

Министерство науки и высшего образования Российской Федерации
ИТМО

Лабораторная работа №3

C++ CLI / FUNCTION / LOOP / RECURSION

по дисциплине:
"C++ и UNIX системы"

Выполнил:
Студент 3 курса, группы Z3300
Заварина Софья

Преподаватель:
Маслов И.Д.

Санкт-Петербург
2026

Цель

Познакомить студента с основными алгоритмическими конструкциями, которые будут использоваться для создания CLI программы. Далее продемонстрировать эффективность использования механизма рекурсии.

C++ алгоритмы: CLI Калькулятор вещественных чисел $+$, $-$, $^$, $.$. Реализация с использованием только функций, условий, циклов, $+$ и $-$. Вид команд в консоли: calc plus / minus / power; Ханойская башня, результат корректной последовательности.

Задача и решение

1. [C++ CLI CALC] Создать программу CALC с интерфейсом CLI.

Создать программу под названием CALC, которая будет принимать на вход 3 аргумента (2 операнда и 1 оператор). Оператор может быть: $+$, $-$, $^$. Реализация операторов только с использованием функций, условий, циклов, $+$, $-$ и $*$.

```
#include <iostream>
#include <string>

long long pow(long long operand_1, long long operand_2) {
    if (operand_1 == 0 and operand_2 == 0) {
        return 1;
    }
    if (operand_2 < 0) {
        std::cout << operand_2 << " must be > 0" << std::endl;
        return 0;
    }
    long long result = 1;
    for (int i = 1; i <= operand_2; i++) {
        result = result * operand_1;
    }
    return result;
}

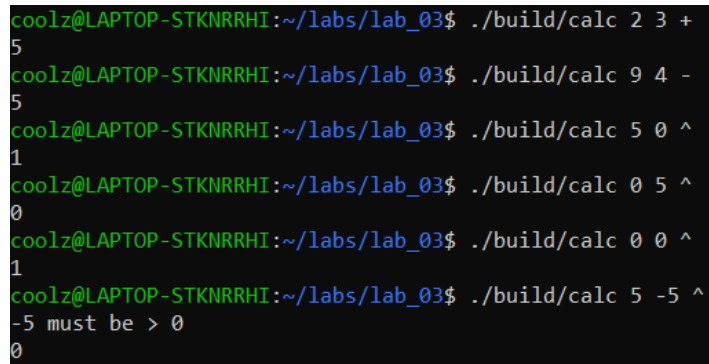
long long calculate(long long operand_1, long long operand_2, char op) {
    if (op == '+') {
        return operand_1 + operand_2;
    }
    if (op == '-') {
        return operand_1 - operand_2;
    }
    if (op == '^') {
        return pow(operand_1, operand_2);
    }
    std::cout << "Error \n" << std::endl;
    return 0;
}

int main(int argc, char* argv[]) {
    if (argc != 4) {
        std::cout << "Incorrect input \n" << std::endl;
        return 1;
    }
}
```

```

    long long a = std::stoll(argv[1]);
    long long b = std::stoll(argv[2]);
    char op = argv[3][0];
    long long result = calculate(a, b, op);
    std::cout << result << std::endl;
    return 0;
}

```



```

coolz@LAPTOP-STKNRRHI:~/labs/lab_03$ ./build/calc 2 3 +
5
coolz@LAPTOP-STKNRRHI:~/labs/lab_03$ ./build/calc 9 4 -
5
coolz@LAPTOP-STKNRRHI:~/labs/lab_03$ ./build/calc 5 0 ^
1
coolz@LAPTOP-STKNRRHI:~/labs/lab_03$ ./build/calc 0 5 ^
0
coolz@LAPTOP-STKNRRHI:~/labs/lab_03$ ./build/calc 0 0 ^
1
coolz@LAPTOP-STKNRRHI:~/labs/lab_03$ ./build/calc 5 -5 ^
-5 must be > 0
0

```

Рис. 1: Работа калькулятора

2. [C++ RECURSION] Решить задачу ханойской башни с использованием рекурсии

Описание: Ханойская башня является одной из популярных головоломок XIX века. Даны три стержня, на один из которых нанизаны восемь колец, причём кольца отличаются размером и лежат меньшее на большем. Задача состоит в том, чтобы перенести пирамиду из восьми колец за наименьшее число ходов на другой стержень. За один раз разрешается переносить только одно кольцо, причём нельзя класть большее кольцо на меньшее.

Результат обнаруженной последовательности шагов записать в виде двусвязного списка. В конце программы сделать вывод этого списка на экран. Освободить память списка перед завершением программы.

```

#include <iostream>

// Узел двусвязного списка
struct Node {
    int disk;    // номер перемещаемого кольца
    int from;    // исходный стержень
    int to;      // целевой стержень
    Node* prev; // указатель на предыдущий ход
    Node* next; // указатель на следующий ход

    Node(int d, int f, int t) : disk(d), from(f), to(t), prev(nullptr), next(nullptr) {}
};

// Добавление элемента в конец списка
void append(Node*& head, Node*& tail, int disk, int from, int to) {
    Node* newNode = new Node(disk, from, to);
    if (!head) {
        head = tail = newNode;
    } else {
        tail->next = newNode;
        newNode->prev = tail;
    }
}

```

```

        tail = newNode;
    }
}

void printList(Node* head) {
    Node* cur = head;
    while (cur) {
        std::cout << "Move disk " << cur->disk << " from " << cur->from
                    << " to " << cur->to << '\n';
        cur = cur->next;
    }
}

// Полное освобождение памяти списка
void clearList(Node*& head, Node*& tail) {
    Node* cur = head;
    while (cur) {
        Node* next = cur->next;
        delete cur;
        cur = next;
    }
    head = tail = nullptr;
}

void hanoy(int n, int from, int to, int temp, Node*& head, Node*& tail) {
    if (n == 1) {
        append(head, tail, 1, from, to);
        return;
    }
    hanoy(n - 1, from, temp, to, head, tail);
    append(head, tail, n, from, to);
    hanoy(n - 1, temp, to, from, head, tail);
}

int main() {
    Node* head = nullptr;
    Node* tail = nullptr;

    hanoy(8, 1, 3, 2, head, tail);

    // Вывод результата
    printList(head);

    // Освобождение памяти
    clearList(head, tail);

    return 0;
}

```

```
Move disk 1 from 1 to 2
Move disk 2 from 1 to 3
Move disk 1 from 2 to 3
Move disk 3 from 1 to 2
Move disk 1 from 3 to 1
Move disk 2 from 3 to 2
Move disk 1 from 1 to 2
Move disk 4 from 1 to 3
Move disk 1 from 2 to 3
Move disk 2 from 2 to 1
Move disk 1 from 3 to 1
Move disk 3 from 2 to 3
Move disk 1 from 1 to 2
Move disk 2 from 1 to 3
Move disk 1 from 2 to 3
Move disk 5 from 1 to 2
```

Рис. 2: Работа обнаружения последовательности

3. [SAVE] Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)

Заключение

В ходе лабораторной работы я написала две программы на C++ — калькулятор и решение Ханойской башни с двусвязным списком.

Калькулятор. Программа принимает два числа и оператор (+, -, ^). Сложение и вычитание — простые, а вот возведение в степень пришлось реализовать самой — циклом умножения.

Ханойская башня. Решила задачу рекурсией для 8 колец — получилось 255 ходов. Каждый ход сохраняется в двусвязный список: узел хранит номер кольца, откуда и куда, плюс указатели prev и next. В конце список выводится на экран, а память освобождается через delete.

В целом работа дала понимание, как работают рекурсивные алгоритмы и структуры данных вроде списков.